FBFISADetector: A Method based on Filter Mechanism to detect the Instruction Set Architecture of Monolithic Firmware

Qing Yang[1,2], Shangru Zhao[2] and Yuqing Zhang[1,2,*]
[1]School of Cyber Engineering, Xidian University, Xi'an, China
yqing@stu.xidian.edu.cn
[2]National Computer Network Intrusion Protection Center, University of Chinese Academy of Sciences, Beijing, China
zhaosr@nipc.org.cn, *zhangyq@ucas.ac.cn

***Abstract*—Firmware identification is a core component of firmware analysis technology, whose primary purpose is to provide sufficient information for subsequent analysis processes. Among such information, *Instruction Set Architecture (ISA)* information is particularly critical – without it, reverse engineering and firmware simulation cannot be carried out. However, currently there is no effective tool for identifying the ISA of monolithic firmware. This paper presents an new identification method based on a filter mechanism to address the issue of ISA identification for monolithic firmware. By studying multiple instances of instruction set architectures, our key observation is that certain instructions appear in pairs, and we summarize the concept of *Instruction Pairs* and develop a mechanism to extract and filter binary blocks, which integrates instruction pairs and the logical correctness analysis of the *P-Code Intermediate Representation (IR)*. Furthermore, we build a prototype of *FBFISADetector*, a monolithic firmware ISA identification system based on the filter mechanism. This system uses the aforementioned methods to extract and filter binary blocks and determines the ISA of tested firmware by the number of remaining binary blocks. Finally, we evaluate our work, and the results show that: In the test of 870 ARM-Cortex M architecture samples from the monolithic firmware dataset, the identification accuracy reaches 98%; in the test of more than 15,000 ELF files covering 5 ISAs (ARM, RISC-V64, MIPS, POWERPC, TRICORE), the overall identification accuracy is as high as 99%. Meanwhile, we verify the code region hit effect of the instruction pair filtering mechanism and the IR analysis filtering mechanism based on the above ELF file dataset. The experimental results demonstrate that the final binary blocks have high credibility regarding their attribution to the current ISA.**

***Index Terms*—ISA identification, binary analysis, reverse engineering, monolithic firmware, internet of things**

I. Introduction

With the development of Internet of Things (IoT) technology, the number of embedded devices is experiencing an explosive growth trend, and the involved fields are also gradually expanding. As software that directly operates on embedded hardware, firmware holds an extremely high position in security analysis, and the related technologies are becoming increasingly diverse, such as firmware static analysis technology [1]–[5], [16], firmware simulation technology [6]–[9], and firmware fuzzing technology [10]–[13].

Firmware analysis strategies are highly similar to binary analysis. Like binary analysis, firmware analysis is also divided into three processing stages: acquisition, identification, and analysis, but there are also differences. For instance, firmware simulation technology is divided into three phases: pre-simulation, simulation setup, and simulation [14]. Among these phases, the identification of firmware information in the pre-simulation phase alone involves the firmware *Instruction Set Architecture (ISA)*, firmware *base address*, firmware program *entry point*, firmware *memory layout*, and so on.

Obtaining ISA information is the starting point for all firmware analysis works. Without ISA information, subsequent identification tasks (such as entry address identification, library function identification, etc.) and analysis work cannot be carried out [15]. Therefore, when firmware's ISA information is lacking, almost all cutting-edge firmware security analysis works (especially for *monolithic firmware* or *non-standard binary file* [30]) choose to conduct analysis under a specified architecture (e.g. ARM-Cortex) to avoid the obstacles caused by such problems [15]–[17].

Due to the complexity of modern computer systems (which contain a large number of heterogeneous devices) and the spread of malware with its strong obfuscation capabilities, some binary security researchers have found bytes frequency analysis methods [19]–[25] to identify ISA of certain executable files. In terms of instruction frequency analysis, machine learning (ML) and deep learning (DL) have demonstrated unique advantages. For example, some machine learning methods have achieved a precision rate of more than 98% in identifying the ISA of executable files [25]. However, due to the inherent flaws of frequency analysis methods and the intrinsic properties of instructions, current ML and DL methods have limitations in identifying the ISA of embedded monolithic firmware: (i) more complex noisy data – binary

* is the corresponding author. This work was supported by the National Natural Science Foundation of China (U2336203), Beijing Natural Science Foundation (L242015, 4242031), and the Innovation Fund of Xidian University(GNYZ2024QC009).

information in monolithic firmware includes not only code and data sections but also a large number of unprogrammed address spaces; (ii) the influence of operands on opcodes; and (iii) lack of feature extraction for relationships between instructions. Furthermore, there also exists an ISA identification method that adopts static analysis [31], but fails to thoroughly explore the patterns of relational features between instructions and essentially still focuses on analyzing the frequency of the instructions related to $call$.

We attempt to use non-frequency analysis methods to identify the ISA of monolithic firmware, aiming to avoid noise from non-code regions and operands. In our research, *the function prologues and epilogues* in *angr*'s archinfo provided us with inspiration, prompting us to study the relationship features between instructions within function bodies in binary files of different instruction set architectures. Through extensive practice, we found that almost all architectures have the feature of instructions appearing in pairs, and these instruction pairs usually indicate certain operations of the current instruction set. For example, instruction pairs involved in stack frame operations include push and pop instructions (ARM's $push$ and pop, MIPS's sw and lw, PowerPC's $stmw$ and lmw); return instructions at the end of almost all main function bodies and common register operation instructions at the front of functions (such as RISC-V 64's $c.addi16sp$ and ret); some other operations in certain architectures also have paired instructions (such as ARM's If-Then operations which include cmp and $b$xx). Based on the above inspirations and findings, we implemented a Binary Block filtering model using instruction pairs and analysis of the data flow state of instruction sequences to achieve ISA identification of binary files with complete semantic content (monolithic firmware, executable files). In particular, to achieve unified processing of instruction sequences across multiple architectures, we used Ghidra's P-Code IR [29] to perform data flow state analysis. We have the following contributions:

- We have studied examples of multiple instruction set architectures and summarized the concept of instruction pairs. In addition, using p-code IR, we implemented the analysis of the logical correctness of an instruction sequence from two aspects: path coverage integrity and register declaration and usage status.
- We have developed a prototype system for ISA identification of monolithic firmware – FBFISADetector. We implemented our filtering mechanism and completed the determination method of ISA through the analysis of instruction pairs and IR.
- We have evaluated the system. We achieve a high accuracy rate ($\sim$98%) in 870 monolithic firmware samples of the ARM-Cortex M architecture and a recognition accuracy of over 99% in 15,000 ELF files from 5 ISAs. Moreover, another experiment indicates that the binary blocks we obtained are highly credible to belong to the current ISA.

The remainder of this paper is organized as follows. **Section II** introduces related work. **Section III** elaborates on our methodology. **Section IV** describes the implementation of *FBFISADetector*, an ISA identification system based on our proposed methodology. **Section V** provides evaluations of our work. **Section VI** concludes our work.

II. Related Work

In this section, we will focus on introducing existing ISA recognition methods, and classify ISA identification methods into two categories: static (binary) analysis methods and ML-based frequency analysis methods.

ISA recognition is a crucial and non-negotiable step in firmware analysis. When analyzing firmwares containing OS, binwalk [18], a static binary analysis tool, is generally the method of choice. The binwalk identifies *magic symbols* within binary files to analyze file structures. Additionally, several existing methods achieve ISA recognition through feature extraction [19]–[25]. [21] pioneered the application of file carving and ML algorithms (from the malware detection domain) to PLC firmwares, concluding that the combination of a byte-value frequency file extraction algorithm and a Support Vector Machine (SVM) algorithm (with feature selection via information gain) yields the optimal performance. Clemens [19] demonstrated that byte-value histograms can preserve certain opcode features, proposing a feature extraction method that converts binary content into byte-value histogram vectors; these vectors are then used as input to a multi-classification model to complete ISA recognition. WiliBin [24] also employs byte histograms for ISA recognition and additionally attempts to use machine learning methods to locate code regions in ELF files. ELISA [20] uses byte histograms for ISA recognition; in addition, it leverages the *known function prologues and epilogues patterns* documented in angr's archinfo [27], appending the number of pattern matches for each architecture to the end of the feature vector to enhance architectural feature representation. [22] proposed an SVM-based ISA recognition method (SVM-IBPS) and compared it with binwalk, concluding that machine learning methods outperform binwalk in both precision and efficiency. ISADetect [23] summarizes previous machine learning-based methods and provides an extensive dataset of ELF files across various ISAs. The Dinuka Sahabandu team [25] argued that for some architectures, most opcodes are shorter than 1 byte, and the use of byte-value histograms leads to the loss of bit-level information in instructions. Thus, they proposed a TF-IDF feature extraction method based on byte encoding, which not only enhances the extraction of ISA opcode features but also reduces interference from semantically irrelevant bytes.

In addition to the aforementioned ML methods, there exists a static analysis method – EDFIR [31]. It adopts a static analysis method, which performs detailed observation and summary of the characteristics of call instructions, and performs an experiment on the ISA identification precision of real firmware.

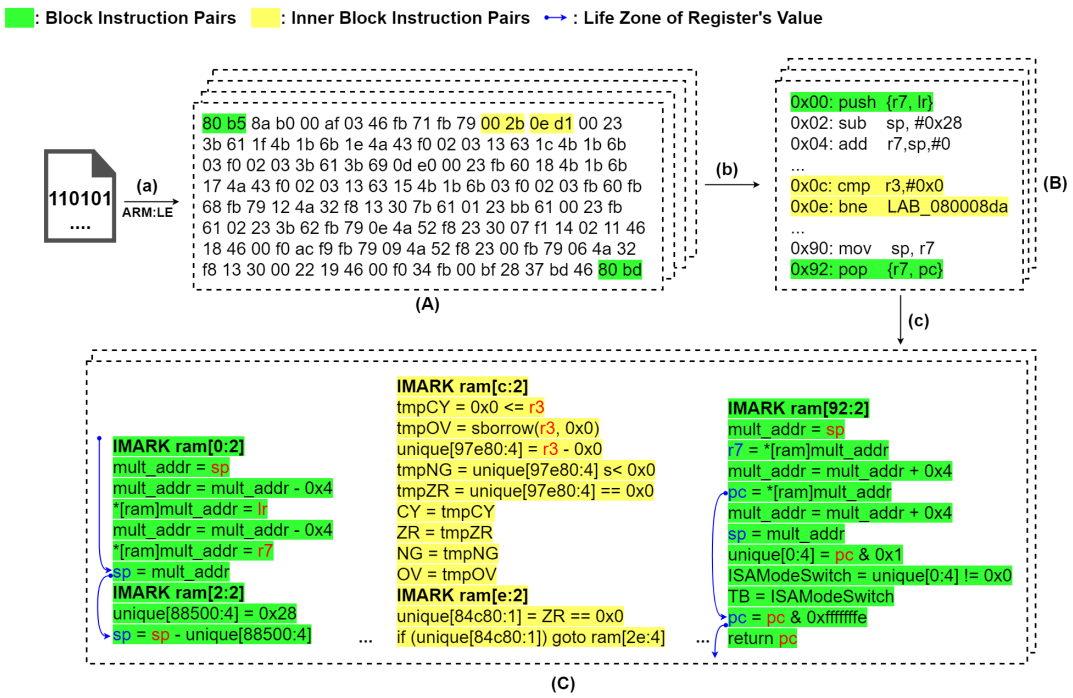


Fig. 1. Overview of Methodology (Taking ARM Little-Endian as an Example). First, we segment binary file by using Block Instruction Pairs of ARM:LE (Fig. 1.(a)). Then, we search for Inner Block Instruction Pairs in binary blocks (Fig. 1.(A)) and filter out binary blocks that do not contain Inner Block Instruction Pairs (Fig. 1.(b)). Next, we check whether the binary blocks can be correctly reverse-engineered (Fig. 1.(B)) and filter out those that cannot be reverse-engineered (Fig. 1.(c)). Finally, we analyze whether the IR logic of the remaining Binary Blocks is correct (Fig. 1.(C)).

III. Methodology

In this section, we will introduce our methodology (See Fig. 1.), which include (i) how we use instruction pairs to segment and screen monolithic firmware or other binary content; (ii) how we use the logic of IR analysis to screen Binary Blocks.

A. Definition and Application of Instruction Pairs

Because within the address range of firmware, not all address spaces represent valid content (e.g., instruction content). To extract valid content from binary firmware as much as possible, we have designed the concept of ***instruction pairs***.

Definition. In a specific ISA, the set of all function structures is denoted as S, and the set of continuous instruction sequence segments within all function structures is denoted as SB. An ins (instruction) consists of the opcode in the instruction and the partial bits of the operand. The following are the concept of instruction pairs that we designed and the two major categories of instruction pairs:

Definition 1 (Characteristic Instruction Pairs): In a sequence of multiple consecutive binary instruction sequences $\{B_0, \ldots, Bn\} \subseteq SB$, there exists a pair of instructions ins_1 and ins_2 that have one or more specific instruction relationships, denoted as $features = pair(ins_1, ins_2)$. For example, identical instruction frequencies (Set $FE \in features$), restricted inter-instruction distance (Set $DL \in features$), and identical register usage (Set $RS \in features$), etc.

Definition 2 (Block Instruction Pairs): In a specific ISA, there exists a category of function structure $S_i \in S$, and $S_iB \subseteq SB$ represents the set of instruction sequences of the function structure S_i. For any instruction subsequence $s_ib_x \subseteq S_iB_x$ where $\forall S_iB_x \in S_iB$, the instructions ins_1 and ins_2 at the two ends (start and end) of s_ib_x form an instruction pair $pair(ins_1, ins_2)$, and both ins_1 and ins_2 appear only once in b. That is, $fe(ins_1, ins_2, s_ib_x, 1) \in FE \in pair(ins_1, ins_2)$, which $fe(ins_1, ins_2, s_ib_x, 1)$ means that ins_1 and ins_2 have the same frequency of occurrence in the instruction sequence s_ib_x, and each of them appears only once.

Definition 3 (Inner Block Instruction Pairs): Within the instruction subsequence s_ib_x, all characteristic instruction pairs other than the block instruction pairs of the function structure S_i are Inner Block Instruction pairs.

Application of Block Instruction Pairs. The main function of block instruction pairs is to extract valid instruction content as accurately as possible. For example, the $push$ $\{lr, rx\}$ and pop $\{pc, rx\}$ instructions in the ARM-Cortex architecture; the sw $ra, local_x(sp)$ and lw $ra, local_x(sp)$ or jr ra instructions in the MIPS-32 architecture. In the binary content of monolithic firmware, there is usually a large amount of data content (e.g., vector tables, data tables) and large areas of unprogrammed address spaces. However, for the task of firmware ISA identification, these data contents and unprogrammed address spaces are noise. To eliminate the interference of such noise on the identification task, a method capable of locating code regions

is crucial. Therefore, we use block instruction pairs to locate the code regions approximately.

Application of Inner Block Instruction Pairs. After obtaining binary blocks that are as meaningful as possible, feature instruction pairs are used to perform auxiliary analysis on the binary blocks again, to identify more accurate valid address spaces (valid binary blocks). For example, in the ARM-Cortex architecture, after we extract possible Binary Blocks using the stack push instruction $push\ \{lr\}$ and $pop\ \{pc\}$, we use the instruction pairs commonly used in conditional jumps, namely $cmp\ rx, ry/\#imm$ and $b$xx $TargetAddress$, to analyze whether if-else related statements are used within the Binary Blocks, and further screen out Binary Blocks with richer semantics.

Based on the above two concepts and application strategies, we can successfully select some binary blocks with rich semantics for further analysis to assist in the identification of firmware ISA. However, it is worth noting that instruction pairs do not imply a strong binding relationship between the occurrences of two instructions, i.e. the occurrence of one instruction must mean the existence of the other. Instruction pairs mean that when two instructions appear simultaneously, they together form a type of semantics/operation, which is why we believe that instruction pairs can screen out Binary Blocks with rich semantic content.

B. Analysis Strategy within Binary Blocks

After block instruction pairs segment the monolithic firmware or other binary file (see Fig. 1.(a)) and inner block instruction pairs screen out binary blocks with rich semantic content (see Fig. 1.(A)), we finally add reverse engineering exploration (see Fig. 1.(B)) and IR logical correctness analysis (see Fig. 1.(C)) for binary block screening. This is to increase the proportion of binary blocks that hit code regions, thereby verifying the credibility of the identification results.

Reverse Engineering Exploration. We use a linear sweep algorithm to perform reverse engineering of the specified architecture (such as ARM:32:LE:Cortex) on binary blocks. The purpose of this method is to prepare the architecture for subsequent IR correctness analysis. In addition, since we know the positions of the instruction pairs within the binary blocks, we can use these positions to judge the reverse engineering results. If a known instruction is inconsistent with the reverse engineering result at the corresponding position, an exception is thrown.

IR Logical Correctness Analysis. To verify the validity of the finally screened Binary Blocks for the current test architecture, we use IR to analyze the content of the binary blocks. IR is used for analysis because it has the advantage of unifying the semantics of machine codes across different architectures. Our method is mainly divided into two aspects:

Path Coverage. After translating Binary Blocks into IR, we traverse them using the Depth-First Search (DFS) algorithm according to their control flow, record all visited varnodes (variable nodes of P-code), and finally check whether there are any unvisited nodes. Specifically, for goto statements that jump to address regions outside the block, we treat them as subfunction calls to prevent the erroneous deletion of correct Binary Blocks. In addition, we also record the declaration status and usage status of registers during the traversal, which are used for subsequent analysis.

Registers State Tracking. To track changes in the register value in detail, we divide the interval from a declaration of a register to its next declaration into a segment, which we call *the Life Zone of Register's Value*. As shown in Fig. 1.(C), within the Life Zone of the value of register rx, we check whether rx is used reasonably. We classify its usage methods into two categories: (i) Direct usage, i.e. being invoked in an expression; (ii) Being used as an argument-passing register in sub-function calls. In addition, in the analyzed binary blocks (functions), some registers are used without being declared. Based on the *Calling Conventions* of various architectures, we have summarized three types of reasonable causes for this situation: (i) Special registers acting as global variables, such as the stack pointer register (sp), program counter (pc/ra), linked register (lr), and other global registers (gp); (ii) Argument Passing registers; (iii) General-purpose registers used for context preservation, such as registers r4-r11 in the ARM 32-Cortex and registers s0-s8 in the MIPS-32.

$$ArchRes(x) = (BlocksNum(x)_{org}, \\ BlocksNum(x)_{inp}, BlocksNum(x)_{ir}) \quad (1)$$

Based on the two aforementioned IR logical analysis methods, we can screen out binary blocks that conform to the characteristics of the specified architecture. After we complete the analysis of all architectures for the file, we can finally send the number of binary blocks screened out by each filter module of each architecture to the final architecture judgment module for the ultimate architecture determination. The screening result of each architecture is represented in the form of a triple (See EquationIII-B), where $ArchRes(x)$ refers to the final returned result of the Instruction Set Architecture x, $BlocksNum(x)_{org}$ denotes the number of binary blocks obtained by segmenting the binary file through block instruction pairs, $BlocksNum(x)_{inp}$ represents the number of binary blocks screened by the inner block instructions pairs (See Fig. 1.(A)), and $BlocksNum(x)_{ir}$ is the number of binary blocks screened by the IR analysis (See Fig. 1.(B) and (C)).

IV. Implements

We designed our ISA identification tool based on the methodology described in III. We mainly designed three modules: Binary Context Pre-processor Module (Section IV-B), Block Filter Module (Section IV-C), and Judge Module (Section IV-D). Fig. 2. describes the workflow of FBFISADetector.

A. Configuration Files

We record the information of each instruction set architecture (such as instruction pairs, supported endianness, and

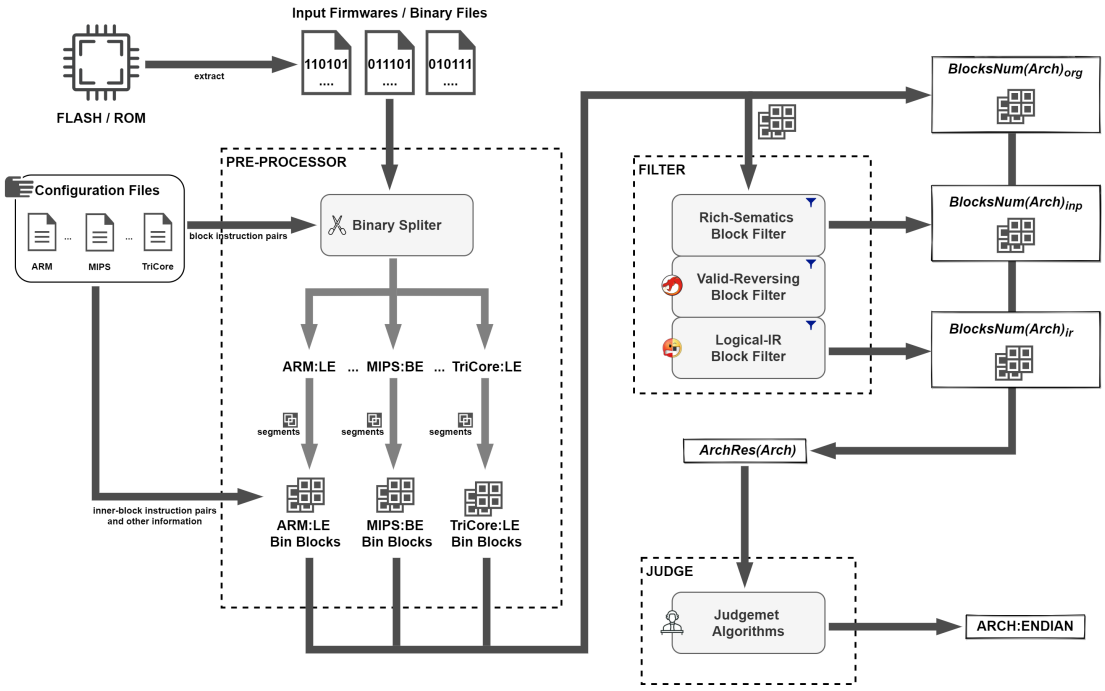


Fig. 2. Overview of FBFISADetector. After obtaining the binary file, we will read the supported ISAs' block instruction pairs and the necessary information of each ISA (e.g., supported endianness, size of endian reverse unit, minimum instruction length, etc.) from the configuration file. And then, we will preprocess the binary file according to the information in the configuration file, splitting it according to different architectures and encapsulating BinaryBlock objects. Next, multiple BinaryBlock objects of different architectures will be sent to the FILTER module for three layers of filtering (Rich-Semantics Block Filter, Valid-Reversing Block Filter, and Logical-IR Block Filter). Finally, we record the screening results of all architecture BinaryBlock objects by each layer of filters and submit them to the Judge module for architecture determination.

register sets) into configuration files according to user manual of different ISA.

*a) **Information of Instruction Pairs**:* As defined in Section III-A, the information of the instruction pair consists of instructions and inter-instruction relationships (features).

Instruction information includes instruction matching patterns and instruction descriptions. In particular, our pattern matching for instructions is not limited to opcodes but includes opcodes and partial bits of operands, which allows the instruction information to contain certain certain characteristic content. For example, the instruction content of $push\ \{r7, lr\}$ under the ARM:BE:32:Cortex architecture is 0xb580, with a big-endian opcode mask of 0xfe00. To retain the information of the lr register, we need to use a mask of 0xff00, so its matching pattern becomes $0\mathrm{x}b5[0\mathrm{x}00 - 0\mathrm{x}ff]$.

Inter-instruction relationship includes two instructions, custom relationship types, and inter-instruction features. Among these, the most critical are the inter-instruction relationships (such as $FE/DL/RS$ in Section III-A). Typically, we use these inter-instruction relationships for Binary Block extraction. For example, if there is a feature RS between $push\ \{r7, lr\}$ and $pop\ \{r7, pc\}$ under the ARM:BE:32:Cortex architecture, we will check whether the register lists in their operand regions are consistent and form an instruction pair based on this condition.

*b) **Other Information**:* Due to differences between different ISAs, information such as supported endianness, word length, and alignment length varies significantly across architectures. For instance, the ARM architecture supports both big-endian and little-endian modes, with endianness reversal performed in half-word (16 bits) units; whereas the TriCore architecture only supports little-endian mode, with endianness reversal performed in word (32 bits) units (instructions shorter than 32 bits are reversed in half-word units). Due to these differences, we choose to record such information in configuration files. Similarly, since different ISAs have different *Calling Conventions*, we record register set information (special registers, general-purpose registers, argument-passing registers, return-value registers, and context-preserving registers) of different ISAs in configuration files to provide information support for subsequent IR analysis.

B. Binary Context Pre-processor Module

The preprocessing module mainly involves segmenting binary content and encapsulating it into blocks. First, we perform pattern matching on the binary content according to the two instructions of the block instruction pairs, following the endianness supported by the specified ISA (e.g., MIPS:BE, ARM:LE, ARM:BE). Then, we assemble the matched two instructions into pairs using Algorithm 1 to segment the binary content. Here, the function $CheckFeature(pair, F)$ returns a boolean value, since it checks whether the instructions within the $pair$ satisfy the characteristic relationship F. Finally, we encapsulate these instructions in *BinaryBlock* objects to

Algorithm 1 Create Block Instruction Pairs

Input: $L1$ (Head Instructions), $L2$ (Tail Instructions), F (Features), ml (minimum length)
Output: $pairs$ (Block Instruction Pairs)

function CREATEBLOCKPAIRS($L1$, $L2$, F, ml)
  Initialize RL (length ml), TL, $pairs$ as empty lists
  if ml is odd **then return** $pairs$
  end if
  for $ins \in L1$ **do**
    $ins.tag \leftarrow 1$
  end for
  for $ins \in L2$ **do**
    $ins.tag \leftarrow 0$
  end for
  $TL \leftarrow L1 \cup L2$, sorted by $ins.start$
  for $ins \in TL$ **do**
    $RL[ins.start \mod ml] \leftarrow ins$
  end for
  for $r \in 0..ml-1$ **do**
    $h \leftarrow null, t \leftarrow null$
    for $ins \in RL[r]$ **do**
      if $ins.tag == 1$ **then**
        $h \leftarrow ins$
      else if $h \neq null$ **then** $t \leftarrow ins$
        if $CheckFeature((h,t), F)$ **then**
          $pairs.add((h,t))$
        end if
        $h \leftarrow null$
      end if
    end for
  end for
  return $pairs$
end function

facilitate subsequent analysis tasks. A *BinaryBlock* records information about block instruction pairs, the start position of the current binary block in the binary content, and the block size.

C. Block Filter Module

To ensure the credibility of our final results, we aim to reduce the number of binary blocks that miss code regions through Filters. We have designed three layers of Filters in the Filter Module:

a) ***Rich-Semantics Block Filter****:* Based on the Inner Block Instruction Pairs provided in the configuration file, we check whether all binary blocks contain any pair of Inner Block Instruction Pairs. If they do, we select them and send them to the next layer of the filter; otherwise, they are discarded.

b) ***Valid-Reversing Block Filter (Based on Ghidra [5])****:* According to the ISA currently being verified and the variants of this ISA supported by Ghidra (angr-pypcode [28]) (e.g., ARM:LE:32:Cortex and ARM:LE:32:v8), we perform reverse engineering on the bytes in the region to which the BinaryBlock object belongs. We verify whether the current variant is suitable using the reverse engineering exploration method described in Section III-B. When a suitable variant is found, we record it and terminate the exploration process. If no suitable variant is found, the current BinaryBlock object is discarded.

This layer mainly serves as preparation for the next layer of the whole filter, and it can be seen that the filtering capability at this layer is not strong. Therefore, the number of remaining BinaryBlocks objects after its filtering is not recorded.

c) ***Logical-IR Block Filter (Based on P-Code)****:* Due to the characteristic of P-Code IR that it can retain the state of machine code to the greatest extent, we choose it for logical analysis. Using the IR logic analysis method described in Section III-B, we verify whether the path coverage is complete and the register usage status is reasonable after the binary content of the current BinaryBlock is translated into IR. In particular, the division of the life zone is based on the execution path rather than the sequential path.

D. Judge Module

Our practice has shown that the Instruction Pairs and IR Analysis methods have good filtering capabilities. Therefore, our architecture determination method is not very complicated.

Hierarchical judgment method based on the number of Binary Blocks. Due to the differences in the content richness of different binary files and the filtering capabilities of instruction pairs across different architectures, we choose to compare the number of binary blocks of each ISA at each layer in a hierarchical manner (See Equation 4).

$$SBN_l = \sum_{k=0}^{n} BlockNum(x_k)_l \tag{2}$$

$$isBestArch(x_i)_l = equal(\max_{k\in\{0,\ldots,n\}} BlockNum(x_k)_l, BlockNum(x_i)_l) \tag{3}$$

$$ISAArch([x_0,\ldots,x_n]) = \begin{cases} x_i, & SBN_{ir} > 0 \ and \ isBestArch(x_i)_{ir} \\ x_j, & SBN_{ir} = 0 \ and \ SBN_{inp} > 0 \\ & \ and \ isBestArch(x_j)_{inp} \\ x_t, & SBN_{ir} = 0 \ and \ SBN_{inp} = 0 \ and \ SBNorg > 0 \\ & \ and \ isBestArch(x_t)_{org} \\ null, & SBN_{ir} = 0 \ and \ SBN_{inp} = 0 \ and \ SBNorg = 0 \end{cases} \tag{4}$$

In equation 2, 3 and 4, $ISAArch([x_0,\ldots,x_n])$ denotes the final output architecture, where n represents the number of supported architectures for identification ($n \geq 0$), and the respective architectures are x_k with $i,j,t,k \in \{0,\ldots,n\}$. SBN_l denotes the total number of all binary blocks output by all supported architectures at the l-th layer of the filter ($l \in \{org, inp, ir\}$).

V. EXPERIMENTS

Our experiments mainly consist of recognition precision verification and component effectiveness analysis experiments (to verify the layers of filter effect).

A. Experiment Setup

We have completed the configuration files for 5 ISAs in total, as shown in Table I. The instruction pairs are represented in the form of a triple *(head_instruction, tail_instruction, features)*. The head instruction refers to the instruction with the lower address in the instruction pair, the tail instruction

TABLE I
INSTRUCTION PAIRS OF 5 ISA IN OUR WORKS

Arch	Bits	Block Instruction Pairs	Inner Block Instruction Pairs
ARM	32	(push-lr,pop-pc,{FE,RS})	(cmp,bx-it,{DL(1)})
RISC-V	64	(c.addi16sp-sub,ret,{FE})	(c.sdsp,c.ldsp,{RS})
MIPS	32	(addiu-sp-sub,blr,{FE})	(lw,bne{RS}), (lw,beq,{RS})
POWERPC	32	(stwu-sp-sub,blr,{FE})	(stmw,lmw,{RS}), (mfspr-lr,mtspr-lr,{RS})
TRICORE	32	(mov.aa-ax-a4,ret,{FE}), (mov-dx-d4,ret,{FE})	(movh.a,lea,{DL(1,5),RS}), (movh.a,ldx,{DL(1),RS}), (movh.a,st.x,{DL(1),RS})

refers to the instruction with the higher address, and the features represent the characteristic relationships between the instructions (see Section III-A).

Our selection of instructions is more meticulous. We represent instructions in the form of *instruction-*{*conditions*}, i.e., instructions that meet certain specific requirements. For example, if a condition includes register names (e.g., sp, lr, pc, ax, dx, rx), it indicates that the operands section of the instruction contains the marker bits of these registers; if the condition is custom-defined, it means that certain bits in the operands area have special meanings. The *it* in the table I represents jumps in the if-then context rather than function calls, while the *sp-sub* represents $sp = sp + (-0\text{x}??)$.

Furthermore, the characteristic relationships between two instructions manifest itself as several verification methods in practical implementation. For example, the FE relationship shared by all block instruction pairs is actually implemented as pattern recognition of instructions and the combination of binary blocks; while $DL(1, x)$ verifies the absolute distance between two instructions, and RS verifies whether the registers marked by the operands of the two instructions are consistent.

We will conduct experimental tests based on the above 5 ISAs. For other ISAs, users can add new configuration files to support the new ISAs. Most of the information in the configuration files can be obtained by applying our instruction pairs theory and combining it with the official manuals of the corresponding ISAs.

B. ISA Recognition Precision

a) ***ISA Recognition precision of Real Monolithic Firmware (ARM Only)****:* We conducted this experiment to verify the ability of our work to identify real monolithic firmware. In terms of the experimental method, we batch-input the monolithic firmwares from the firmware dataset into FBFISADetector for ISA recognition. We recorded the recognition results of the tool and checked whether the result was ARM:LE. If it was, we considered the recognition correct; otherwise, we deemed it a recognition failure.

Dataset. The dataset of Real Monolithic Firmware is sourced from the ARM-Cortex-M monolithic firmwares collected by the UCSB Laboratory [26]. It contains a total of 870 monolithic firmwares, including 797 from FirmXRay [16], 25 from FuzzWare [12], 10 from PolyPyus [17], and 38 provided in other papers. These firmwares cover multiple application fields such as Bluetooth, gateways, PLCs (Programmable Logic Controllers), and drones.

TABLE II
ISA RECOGNITION ACCURACY OF REAL MONOLITHIC FIRMWARES (ARM-CORTEX)

Data Source	Arch:Endian	Number of Firmwares	Number of Correct Recognitions	Precision	Accuracy
FIRMXRAY	ARM:LE	797	787	98.75%	-
FUZZWARE	ARM:LE	25	25	100%	-
POLYPYUS	ARM:LE	10	10	100%	-
OTHERS	ARM:LE	38	38	100%	-
TOTAL	ARM:LE	870	860	-	98.85%

TABLE III
ISA RECOGNITION PRECISION OF MULTI-ARCHITECTURE ELF FILES AND COMPARISON WITH ISADETECT

Data Source	Arch:Endian	Number of ELF Files	Recall	F1-Score	Precision /Accuracy	Precision/Accuracy of ISADetect (RandomForest)
ISADetect/armhf	ARM:LE	3,995	99.97%	99.96%	99.95%	88.12% (=2,648/3,005)
ISADetect/riscv64	RISCV64:LE	4,440	100%	99.99%	99.86%	81.70% (=2,455/3,005)
ISADetect/mips	MIPS:BE	3,565	100%	100%	100%	97.77% (=2,937/3,004)
ISADetect/powerpc	POWERPC:BE	3,631	100%	100%	100%	34.34% (=1,032/3,005)
AURIX/EXAMPLES	TRICORE:LE	312	99.68%	99.68%	99.68%	-
TOTAL	-	15,943	-	-	99.94%	75.48% (=9,072/12,019)

Result Analysis. As can be seen from Table II, the overall ISA recognition accuracy of our work for monolithic firmwares reaches 98.85%. Among all datasets, only the FirmXRay dataset (mainly Nordic bluetooth firmware) has 10 false positives, including null*4, TriCore:LE*5, and ARM:BE*1; the recognition precision of all other firmwares reaches 100%. We inspected the false-positive firmwares in the FirmXRay dataset and found that these firmwares exhibit one or more of the following characteristics: (i) The code sections of the firmware are inadequate, resulting in limited semantic richness; (ii) Even when the ARM-Cortex architecture is selected in Ghidra [5], the code sections of the firmware cannot be successfully reverse-engineered.

b) ***ISA Recognition for Multi-Architecture ELF Files****:* To verify the multi-architecture ISA recognition capability of our work, we need to validate its recognition precision for different architectures. The experimental method is similar to that of real firmwares, except that our test architectures are not limited to the ARM alone. Additionally, to verify the superiority of our method, we have conducted a comparative experiment with ISADetect (with random forest model) – the only open-source method of its kind. However, ISADetect has not made its preprocessing source code public; therefore, we only present the test set precision for relevant ISAs (i.e. ARM, RISC-V64, MIPS, POWERPC). In the test set, the number of test samples for each architecture is approximately 3005.

Dataset. We used ELF files of 4 ISAs (ARM, RISC-V64, MIPS, POWERPC) from the ISADetect dataset (sourced from Debian Linux repositories), as well as the compiled results (.elf files) of TriCore project samples provided in AURIX IDE. Among these, the ISADetect dataset provides 3,500 to 4,500 pieces of data for each ISA, and we compiled 312 project samples in AURIX IDE.

Result Analysis. Our work achieves an overall ISA recognition accuracy of 99.94%, while the state-of-the-art machine learning method [25] has an precision of less than 99% for 5-architecture recognition. Among the tested datasets, there are a small number of false positives in ISADetect/armhf, ISADetect/riscv64, and AURIX/EXAMPLES, whereas the recog-

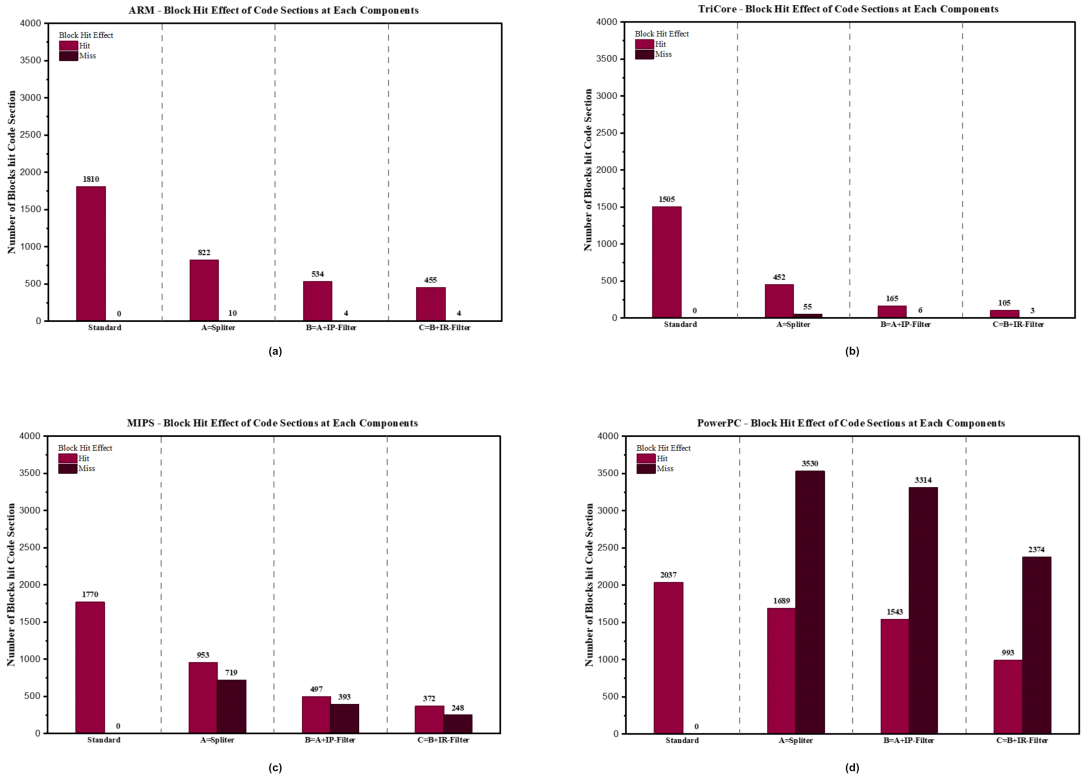


Fig. 3. Validation of each Layer Effectiveness of Filter. Standard represents the number of sections extracted from ELF files; A represents the number of binary blocks encapsulated by the Spliter of FBFISADetector; B represents the number of binary blocks filtered out by the Rich-Semantics Blocks Filter; C represents the number of binary blocks finally filtered out by the entire Filter module.

nition precision of ISADetect/mips and ISADetect/powerpc reaches 100%. Furthermore, it can be seen from the comparative experiments that the precision of our method is significantly higher than that of ISADetect.

In the ISADetect dataset, our false positives typically originate from ELF files with very small sizes, which is consistent with the results from the real firmware testing section. For the TriCore architecture, the false-positive test file (named *iLLD_TC375_ADS_Bluetooth_RFCOMM.elf*) contains a large amount of dense binary code in its data section (labeled *g_minidriverBinary*) that resembles ARM:LE binary code. We speculate that this may be driver programs for certain peripherals.

C. Validation of Component Effectiveness

Since the block instruction pairs and inner-block instruction pairs selected for each ISA are different, we need to analyze two aspects of our work: the impact of instruction pair selection on components, and the mutual influence between components.

We extracted the positions of sections from ELF files and analyzed whether the binary blocks extracted by each component of FBFISADetector could hit these sections. However, sections in ELF files may contain data segments. Although we made every effort to eliminate some small data tables, large data tables were still retained.

Dataset. Due to incomplete ELF file header information in ISADetect/riscv64, we only tested the remaining four architectures (ARM, MIPS, POWERPC, TRICORE). For ease of observation, we controlled the number of ELF files for each architecture to ensure that the total number of function bodies (binary blocks) in the ELFs falls within the range of 1,500 to 2,100. Regarding the dataset sources: ARM uses some ELF files from the UCSB monolithic firmware dataset; MIPS and POWERPC data come from some ELF files in the ISADetect dataset; TriCore data are derived from some ELF files compiled from AURIX IDE project samples.

Result Analysis. Overall (see Fig. 3.), each of our components – Rich-Semantics Block Filter (IP-Filter) and Logical-IR Block Filter (IR-Filter) – exhibits strong filtering capability for binary blocks in the unmatched code regions. In the experiment, we observe that for ARM, TriCore, and MIPS, the number of binary blocks extracted by the components of the FBFISADetector that hit the code regions is greater than that of those that miss. However, the opposite is true for PowerPC. After analysis, we believe that the ELF files in ISADetector/powerpc have incomplete header information. This issue results in the following outcome: although we can correctly identify the architecture, many binary blocks cannot be properly classified into code regions, which also explains why the hit rate for PowerPC is also low. In addition, we also observed that the rate of change for PowerPC from A

to B is not significant. This is mainly because the inner-block instruction pairs we selected for PowerPC represent the storage and retrieval of context content, and their occurrence frequency highly overlaps with that of the block instruction pairs. This also means that its inner-block instructions lack sufficient capability to screen for binary blocks with rich semantics.

VI. Conclusion

This study focuses on addressing the core challenge of accurate and efficient instruction set architecture (ISA) identification for monolithic firmware. First, We summarized the concept of instruction pairs, and leveraged this finding along with IR logic verification to design our ISA identification system—FBFISADetector (Filter-Based Firmware ISA Detector). Furthermore, we have proven through experiments the system's high precision and reliability in ISA identification for Monolithic Firmware. Finally, we also identified a limitation of the system: its relatively poor capability in ISA identification for small binary files (i.e., files with insufficient binary semantic content), and we recommend frequency analysis method (e.g. LLM) as a more suitable strategy to address this extreme situation.

References

[1] Costin Andrei, Jonas Zaddach, Aurélien Francillon, and Davide Balzarotti. "A Large-scale analysis of the security of embedded firmwares." In 23rd USENIX security symposium (USENIX Security 14), pp. 95-110. 2014.

[2] Cheng Kai, Qiang Li, Lei Wang, Qian Chen, Yaowen Zheng, Limin Sun, and Zhenkai Liang. "DTaint: detecting the taint-style vulnerability in embedded device firmware." In 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 430-441. IEEE, 2018.

[3] Ying Huan, Yanmiao Zhang, Lifang Han, Yushi Cheng, Jiyuan Li, Xiaoyu Ji, and Wenyuan Xu. "Detecting buffer-overflow vulnerabilities in smart grid devices via automatic static analysis." In 2019 IEEE 3rd Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), pp. 813-817. IEEE, 2019.

[4] Hex Rays. [n.d.]. Retrieved from https://hex-rays.com/products/ida/.

[5] NSA. [n.d.]. Ghidra. Retrieved from https://ghidra-sre.org/.

[6] Kruegel Christopher. "Full system emulation: Achieving successful automated dynamic analysis of evasive malware." In Proc. BlackHat USA Security Conference, pp. 1-7. 2014.

[7] Zaddach Jonas, Luca Bruno, Aurelien Francillon, and Davide Balzarotti. "AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares." In NDSS, vol. 14, no. 2014, pp. 1-16. 2014.

[8] Gustafson Eric, Marius Muench, Chad Spensky, Nilo Redini, Aravind Machiry, Yanick Fratantonio, Davide Balzarotti et al. "Toward the analysis of embedded firmware through automated re-hosting." In 22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019), pp. 135-150. 2019.

[9] Clements Abraham A., Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. "HALucinator: Firmware re-hosting through abstraction layer emulation." In 29th USENIX Security Symposium (USENIX Security 20), pp. 1201-1218. 2020.

[10] Feng Bo, Alejandro Mera, and Long Lu. "P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling." In 29th USENIX Security Symposium (USENIX Security 20), pp. 1237-1254. 2020.

[11] Zhou Wei, Le Guan, Peng Liu, and Yuqing Zhang. "Automatic firmware emulation through invalidity-guided knowledge inference." In 30th USENIX Security Symposium (USENIX Security 21), pp. 2007-2024. 2021.

[12] Scharnowski Tobias, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. "Fuzzware: Using precise MMIO modeling for effective firmware fuzzing." In 31st USENIX Security Symposium (USENIX Security 22), pp. 1239-1256. 2022.

[13] Chesser Michael, Surya Nepal, and Damith C. Ranasinghe. "MultiFuzz: A Multi-Stream Fuzzer For Testing Monolithic Firmware." In 33rd USENIX Security Symposium (USENIX Security 24), pp. 5359-5376. 2024.

[14] Wright Christopher, William A. Moeglein, Saurabh Bagchi, Milind Kulkarni, and Abraham A. Clements. "Challenges in firmware re-hosting, emulation, and analysis." ACM Computing Surveys (CSUR) 54, no. 1 (2021): 1-36.

[15] Gritti Fabio, Fabio Pagani, Ilya Grishchenko, Lukas Dresel, Nilo Redini, Christopher Kruegel, and Giovanni Vigna. "Heapster: Analyzing the security of dynamic allocators for monolithic firmware images." In 2022 IEEE Symposium on Security and Privacy (SP), pp. 1082-1099. IEEE, 2022.

[16] Wen Haohuang, Zhiqiang Lin, and Yinqian Zhang. "Firmxray: Detecting bluetooth link layer vulnerabilities from bare-metal firmware." In Proceedings of the 2020 ACM SIGSAC conference on computer and communications security, pp. 167-180. 2020.

[17] Friebertshäuser, Jan, Florian Kosterhon, Jiska Classen, and Matthias Hollick. "Polypyus–the firmware historian." In Workshop on Binary Analysis Research (BAR), vol. 2021, no. 21. 2021.

[18] Binwalk. [Online]. Available: https://github.com/ReFirmLabs/binwalk.

[19] Clemens John. "Automatic classification of object code using machine learning." Digital Investigation 14 (2015): S156-S162.

[20] De Nicolao Pietro, Marcello Pogliani, Mario Polino, Michele Carminati, Davide Quarta, and Stefano Zanero. "ELISA: ELiciting ISA of raw binaries for fine-grained code and data separation." In International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, pp. 351-371. Cham: Springer International Publishing, 2018.

[21] K. A. Sickendick, "File carving and malware identification algorithms applied to firmware reverse engineering," Air Force Inst. Technol., Wright-Patterson Air Force Base, OH, USA, Tech. Rep. AFIT-ENG-13-M-46, 2013.

[22] Y. Ma, L. Han, H. Ying, S. Yang, W. Zhao, and Z. Shi, "SVM-based instruction set identification for grid device firmware," in Proc. IEEE 8th Joint Int. Inf. Technol. Artif. Intell. Conf. (ITAIC), May 2019, pp. 214–218.

[23] Kairajärvi Sami, Andrei Costin, and Timo Hämäläinen. "ISAdetect: Usable automated detection of CPU architecture and endianness for executable binary files and object code." In Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy, pp. 376-380. 2020.

[24] B. Beckman and J. Haile, "Binary analysis with architecture and code section detection using supervised machine learning," in Proc. IEEE Secur. Privacy Workshops (SPW), May 2020, pp. 152–156.

[25] Sahabandu Dinuka, J. Sukarno Mertoguno, and Radha Poovendran. "A natural language processing approach for instruction set architecture identification." IEEE Transactions on Information Forensics and Security 18 (2023): 4086-4099.

[26] Monolithic firmware collection. [Github]. https://github.com/ucsb-seclab/monolithic-firmware-collection.

[27] Shoshitaishvili Yan, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen et al. "Sok:(state of) the art of war: Offensive techniques in binary analysis." In 2016 IEEE symposium on security and privacy (SP), pp. 138-157. IEEE, 2016.

[28] Angr-pypcode. [Github]. https://github.com/angr/pypcode

[29] P-Code. [Online]. https://spinsel.dev/assets/2020-06-17-ghidra-brainfuck-processor-1/ghidra_docs/language_spec/html/pcoderef.html

[30] Benkraouda Hadjer, Nirav Diwan, and Gang Wang. "You Can't Judge a Binary by Its Header: Data-Code Separation for Non-Standard ARM Binaries Using Pseudo Labels." In 2025 IEEE Symposium on Security and Privacy (SP), pp. 3727-3745. IEEE, 2025.

[31] JIA Fan, YIN Xiaokang, GAI Xianzhe, CAI Ruijie, LIU Shengli. Function-call Instruction Characteristic Analysis Based Instruction Set Architecture Recognization Method for Firmwares[J]. Computer Science, 2024, 51(6): 423-433. https://doi.org/10.11896/jsjkx.230500087